# Lab 10: Testing and Deployment

Week 10 Assignment (16.03–22.03.2026)

Student Name: _____ Date: _____

## Overview

**Total Points:** 100 (50 points per lab)
**Required Reading:** *React and React Native, Fifth Edition* by Sakhniuk & Boduch (Packt 2024)
**Chapter 11 (Ch. 10):** "Unit Testing in React" (pp. 175–195)
**Sections to read:** Why test? (p. 176), Jest basics (p. 178), React Testing Library (p. 184), Testing components (p. 188)

### Key Vocabulary

- **Unit Testing**: Testing individual units of code in isolation (Ch. 11, p. 176).
- **Jest**: JavaScript testing framework with mocking and assertion capabilities (Ch. 11, p. 178).
- **React Testing Library (RTL)**: Library for testing React components by querying the DOM (Ch. 11, p. 184).
- **fireEvent**: RTL function to simulate user events (Ch. 11, p. 186).
- **screen**: RTL object for querying rendered elements (Ch. 11, p. 185).
- **Mocking**: Replacing real implementations with fake ones for testing (Ch. 11, p. 180).
- **Coverage**: Percentage of code exercised by tests.

### Testing Pyramid

| Type | Description |
|---|---|
| E2E | End-to-end tests from user perspective |
| Integration | Tests that verify multiple units work together |
| Unit | Tests individual functions/-components in isolation |
| Static | Type checking and linting (TypeScript, ESLint) |

**Best Practice:** Many unit tests at the base, fewer E2E tests at the top.

Testing Best Practices (Ch. 11)

- Test behavior, not implementation
- Write tests that resemble how users interact with your app
- Avoid testing internal implementation details
- Aim for meaningful assertions, not just "renders without crashing"

# 1. Lab 10.1: Unit Testing with Jest and React Testing Library (50 Points)

## Objective

Learn to write unit tests for React components using Jest and React Testing Library. Students will understand testing best practices and how to test components in isolation. This aligns with Ch. 11, "Unit Testing in React" (pp. 175–195).

## Background

Ch. 11 explains that testing is crucial for maintaining code quality. Jest provides the testing infrastructure, while React Testing Library helps test components from the user's perspective by querying the DOM.

**Key Testing Library Functions (Ch. 11):**

- `render`: Renders a component to the DOM
- `screen`: Queries the rendered output
- `fireEvent`: Simulates user interactions
- `cleanup`: Removes rendered components

## Problem Statement

Build and test a Todo application:

- TodoList component with add/delete functionality
- Write tests for all component behaviors
- Achieve good test coverage
- Practice RTL best practices

## Tasks

### Task 1: Setup Testing Environment (10 points)

1. Create a Vite React TypeScript project with testing:

```
npm create vite@testing-app -- --template react-ts
cd testing-app && npm install
npm install -D @testing-library/react @testing-library/jest-dom
    @testing-library/user-event jest @types/jest ts-jest
```

2. Configure Jest in `jest.config.js`:

```
export default {
  preset: 'ts-jest',
  testEnvironment: 'jsdom',
  setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
  moduleNameMapper: {
    '\\.(css|less|scss|sass)$': 'identity-obj-proxy'
  },
  transform: {
```

```
    '^.+\\.tsx?$': ['ts-jest', { useESM: true }]
  }
};
```

3. Create `jest.setup.ts`:

```
import '@testing-library/jest-dom';
```

4. Reference: Ch. 11, p. 178 - Jest setup.

### Task 2: Create TodoList Component (15 points)

1. Create `src/components/TodoList.tsx`:

```
import { useState } from "react";

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

interface TodoListProps {
  initialTodos?: Todo[];
}

export function TodoList({ initialTodos = [] }: TodoListProps) {
  const [todos, setTodos] = useState<Todo[]>(initialTodos);
  const [newTodo, setNewTodo] = useState("");

  const addTodo = () => {
    if (newTodo.trim()) {
      setTodos([
        ...todos,
        { id: Date.now(), text: newTodo.trim(), completed: false }
      ]);
      setNewTodo("");
    }
  };

  const toggleTodo = (id: number) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    ));
  };

  const deleteTodo = (id: number) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  const handleKeyDown = (e: React.KeyboardEvent) => {
    if (e.key === "Enter") {
      addTodo();
    }
  };

  return (
    <div className="todo-list">
      <h1>Todo List</h1>
```

```
      <div className="todo-input-container">
        <input
          type="text"
          data-testid="todo-input"
          value={newTodo}
          onChange={(e) => setNewTodo(e.target.value)}
          onKeyDown={handleKeyDown}
          placeholder="Add a new todo..."
        />
        <button data-testid="add-button" onClick={addTodo}>Add</button>
      </div>

      <ul data-testid="todo-list">
        {todos.map(todo => (
          <li
            key={todo.id}
            data-testid="todo-item"
            className={todo.completed ? "completed" : ""}
          >
            <input
              type="checkbox"
              data-testid="todo-checkbox"
              checked={todo.completed}
              onChange={() => toggleTodo(todo.id)}
            />
            <span data-testid="todo-text">{todo.text}</span>
            <button
              data-testid="delete-button"
              onClick={() => deleteTodo(todo.id)}
            >
              Delete
            </button>
          </li>
        ))}
      </ul>

      <div data-testid="todo-count">
        {todos.length} todos ({todos.filter(t => t.completed).length} completed)
      </div>
    </div>
  );
}
```

2. Reference: Ch. 11, p. 188 - Testing components.

3. Uses data-testid attributes for reliable queries.

### Task 3: Write Tests (25 points)

1. Create `src/components/TodoList.test.tsx`:

```
import { render, screen, fireEvent } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { TodoList } from "./TodoList";

describe("TodoList Component", () => {
  describe("Rendering", () => {
    test("renders empty todo list", () => {
      render(<TodoList />);
      expect(screen.getByText("Todo List")).toBeInTheDocument();
      expect(screen.getByText("0 todos (0 completed)")).toBeInTheDocument();
    });
```

```
  test("renders with initial todos", () => {
    const initialTodos = [
      { id: 1, text: "Test todo", completed: false }
    ];
    render(<TodoList initialTodos={initialTodos} />);

    expect(screen.getByText("Test todo")).toBeInTheDocument();
    expect(screen.getByText("1 todos (0 completed)")).toBeInTheDocument();
  });
});

describe("Adding todos", () => {
  test("adds a new todo via button click", async () => {
    const user = userEvent.setup();
    render(<TodoList />);

    const input = screen.getByTestId("todo-input");
    const addButton = screen.getByTestId("add-button");

    await user.type(input, "New todo item");
    await user.click(addButton);

    expect(screen.getByText("New todo item")).toBeInTheDocument();
    expect(input).toHaveValue("");
  });

  test("adds a new todo via Enter key", async () => {
    const user = userEvent.setup();
    render(<TodoList />);

    const input = screen.getByTestId("todo-input");
    await user.type(input, "Press Enter todo{Enter}");

    expect(screen.getByText("Press Enter todo")).toBeInTheDocument();
  });

  test("does not add empty todo", async () => {
    const user = userEvent.setup();
    render(<TodoList />);

    const input = screen.getByTestId("todo-input");
    const addButton = screen.getByTestId("add-button");

    await user.click(addButton);

    expect(screen.queryAllByTestId("todo-item")).toHaveLength(0);
  });
});

describe("Toggling todos", () => {
  test("toggles todo completion status", async () => {
    const initialTodos = [{ id: 1, text: "Test", completed: false }];
    render(<TodoList initialTodos={initialTodos} />);

    const checkbox = screen.getByTestId("todo-checkbox");
    await userEvent.click(checkbox);

    const todoItem = screen.getByTestId("todo-item");
    expect(todoItem).toHaveClass("completed");
  });
});
```

```
  describe("Deleting todos", () => {
    test("deletes a todo", async () => {
      const initialTodos = [
        { id: 1, text: "Delete me", completed: false }
      ];
      render(<TodoList initialTodos={initialTodos} />);

      const deleteButton = screen.getByTestId("delete-button");
      await userEvent.click(deleteButton);

      expect(screen.queryByText("Delete me")).not.toBeInTheDocument();
    });
  });

  describe("Counter", () => {
    test("updates counter when adding todos", async () => {
      const user = userEvent.setup();
      render(<TodoList />);

      const input = screen.getByTestId("todo-input");
      const addButton = screen.getByTestId("add-button");

      await user.type(input, "Todo 1");
      await user.click(addButton);

      expect(screen.getByTestId("todo-count")).toHaveTextContent("1 todos (0
    completed)");
    });
  });
});
```

2. Reference: Ch. 11, p. 186 - fireEvent and user-event.
3. Uses userEvent for more realistic interactions.

### Deliverable

- React project with testing setup
- `src/components/TodoList.tsx` - Component
- `src/components/TodoList.test.tsx` - Tests
- Jest configuration
- README with test commands and coverage report

**Lab 10.1 Assessment Rubric (50 points)**

- **Task 1 — Setup (10 pts)**: Jest configured (5), RTL installed (5).
- **Task 2 — Component (15 pts)**: Works correctly (10), has testids (5).
- **Task 3 — Tests (25 pts)**: Renders correctly (5), add functionality (5), toggle (5), delete (5), counter (5).

**Grading Notes:** No user-event usage: **-5 pts**. Shallow rendering instead of RTL: **-10 pts**.

## 2. Lab 10.2: Deployment with Vercel/Netlify (50 Points)

### Objective

Deploy a React application to production using modern deployment platforms. Learn CI/CD basics and how to configure build settings for optimal performance.

### Background

Ch. 11 explains that modern deployment platforms like Vercel and Netlify make it easy to deploy React applications. They handle build optimization, CDN distribution, and provide preview deployments for pull requests.

**Deployment Options (Ch. 11):**

- **Vercel**: Optimized for Next.js but works with any React app
- **Netlify**: Drag-and-drop or Git integration
- **GitHub Pages**: Free static hosting
- **AWS Amplify**: Full AWS integration

### Problem Statement

Deploy the Todo application and configure CI/CD:

- Configure build for production
- Deploy to Vercel or Netlify
- Set up GitHub Actions for automated testing
- Configure environment variables

### Tasks

#### Task 1: Configure Production Build (15 points)

1. Update `vite.config.ts` for production:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  build: {
    outDir: 'dist',
    sourcemap: false,
    minify: 'terser',
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
        },
      },
    },
  },
  server: {
    port: 3000,
  },
});
```

2. Create `.env.production`:

```
VITE_API_URL=https://api.example.com
VITE_APP_VERSION=1.0.0
```

3. Create `.gitignore`:

```
node_modules/
dist/
.env
.env.local
.env.*.local
coverage/
*.log
```

4. Reference: Ch. 11, p. 192 - Production builds.

**Task 2: Deploy to Vercel (20 points)**

1. Create `vercel.json`:

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ],
  "headers": [
    {
      "source": "/assets/(.*)",
      "headers": [
        { "key": "Cache-Control", "value": "public, max-age=31536000,
    immutable" }
      ]
    }
  ]
}
```

2. Deploy using Vercel CLI:

```
npm install -g vercel
vercel login
vercel
```

3. Or connect GitHub repo in Vercel dashboard:

   (a) Go to vercel.com/dashboard
   (b) Click "New Project"
   (c) Import from GitHub
   (d) Configure build settings
   (e) Deploy

4. Reference: Ch. 11, p. 194 - Vercel deployment.

### Task 3: Setup GitHub Actions CI (15 points)

1. Create `.github/workflows/ci.yml`:

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    name: Test
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test -- --coverage

      - name: Upload coverage
        uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/

  build:
    name: Build
    runs-on: ubuntu-latest
    needs: test

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Upload build artifacts
```

```
      uses: actions/upload-artifact@v4
      with:
        name: dist
        path: dist/
```

2. Reference: Ch. 11, p. 190 - CI/CD setup.
3. Tests must pass before build runs.

### Deliverable

- Production build configuration
- Vercel deployment (provide URL)
- GitHub Actions workflow
- README with deployment instructions
- Live URL of deployed app

**Lab 10.2 Assessment Rubric (50 points)**

- **Task 1 — Build Config (15 pts)**: Vite config optimized (8), environment variables (7).
- **Task 2 — Deployment (20 pts)**: Deployed successfully (10), URL provided (5), configuration correct (5).
- **Task 3 — CI/CD (15 pts)**: GitHub Actions workflow (10), tests run automatically (5).

**Grading Notes:** No live URL: **-10 pts**. Broken CI workflow: **-5 pts**.

## Submission Requirements and Regulations

### Git Discipline (30 points)

- **Folder Structure (10 points)**: Use `Lab_10/task1/`, `Lab_10/task2/`. Flat dumps: **-10 points**.
- **Conventional Commits (10 points)**: Use `feat:`, `fix:`, `refactor:`. Bad messages: **-5 points**.
- **Incremental History (10 points)**: At least 3 meaningful commits per task. Single dump: **-10 points**.

### Code Quality Standards (20 points)

- **ESLint / TypeScript (10 points)**: No TS errors. Major issues: **-5 points**.
- **Documentation (10 points)**: README explains testing and deployment. Missing: **-10 points**.

### AI Usage Policy

If you use AI tools:

- Submit `AI_REPORT` with: tool used, prompts, how you modified/verify code, what you learned.
- Missing report when AI used: **-50 points**. Incomplete report: **-20 points**.

## Submission Instructions

1. **Lab 10.1**: Jest/RTL testing setup with passing tests.
2. **Lab 10.2**: Deployed application with CI/CD.
3. Use `Lab_10/task1/`, `Lab_10/task2/` folder structure.
4. Include name, student ID, date in README files.
5. Include live URL for deployed app.
6. Submit via course portal.

## Comprehensive Assessment Summary

| Category | Points | Assessment |
|---|---|---|
| Lab 10.1 Tasks | 50 | See Lab 10.1 Rubric |
| Lab 10.2 Tasks | 50 | See Lab 10.2 Rubric |
| Git: Folder structure | 10 | [YES] / [NO] (-10) |
| Git: Conventional commits | 10 | [YES] / [NO] (-5) |
| Git: Incremental history | 10 | [YES] / [NO] (-10) |
| Code quality | 10 | [YES] / [NO] (-5) |
| Documentation | 10 | [YES] / [NO] (-10) |
| **TOTAL** | **100** | |

# Appendix A: Jest Testing Deep Dive

**Jest Configuration Options:**

```
// jest.config.js
export default {
  // Test environment
  testEnvironment: 'jsdom', // For React
  // testEnvironment: 'node', // For Node.js

  // File patterns to test
  testMatch: ['**/__tests__/**/*.test.ts?(x)'],

  // Coverage thresholds
  coverageThreshold: {
    global: {
      branches: 70,
      functions: 70,
      lines: 70,
      statements: 70
    }
  },

  // Module path aliases
  moduleNameMapper: {
    '^@/(.*)$': '<rootDir>/src/$1'
  },

  // Setup files
  setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],

  // Transform files
  transform: {
    '^.+\\.tsx?$': ['ts-jest', { useESM: true }]
  }
};
```

**Common Jest Matchers:**

| Matcher | Use Case |
|---|---|
| toBe(value) | Primitive value equality |
| toEqual(object) | Object/array equality (deep) |
| toBeInTheDocument() | Element exists in DOM |
| toHaveClass(className) | Element has CSS class |
| toHaveTextContent(text) | Element contains text |
| toHaveValue(value) | Input has value |
| toHaveBeenCalled() | Function was called |
| toHaveBeenCalledWith(arg) | Function called with args |
| toBeTruthy() | Value is truthy |
| toBeFalsy() | Value is falsy |
| toThrow() | Function throws error |

**Setup and Teardown:**

```
describe('Component', () => {
  // Run before all tests in describe
  beforeAll(() => {
    // Setup once
  });

  // Run before each test
  beforeEach(() => {
    // Reset state
  });

  // Run after each test
  afterEach(() => {
    cleanup(); // Clean up RTL renders
  });

  // Run after all tests
  afterAll(() => {
    // Cleanup global resources
  });

  test('example', () => {
    // Test code
  });
});
```

### Testing Async Code:

```
// With async/await
test('fetches data', async () => {
  const data = await fetchData();
  expect(data).toEqual(expected);
});

// With async/await and findBy
test('shows loaded data', async () => {
  render(<DataComponent />);
  const element = await screen.findByText('Loaded Data');
  expect(element).toBeInTheDocument();
});

// Testing error states
test('shows error on failure', async () => {
  jest.spyOn(api, 'fetchData').mockRejectedValue(new Error('Failed'));

  render(<DataComponent />);

  expect(await screen.findByText(/error/i)).toBeInTheDocument();
});
```

# Appendix B: React Testing Library Best Practices

### Query Priority (Ch. 11):

| Priority | Query | Use When |
|---|---|---|
| 1 | getByRole | Always preferred |
| 2 | getByLabelText | Form fields |
| 3 | getByPlaceholderText | Input placeholders |
| 4 | getByText | Non-interactive elements |
| 5 | getByTestId | Last resort |

### Why getByRole is Best:

```
// Good - uses role
const button = screen.getByRole('button', { name: 'Submit' });
const heading = screen.getByRole('heading', { level: 1 });

// Okay - uses label
const input = screen.getByLabelText('Email');

// Avoid - fragile
const div = screen.getByTestId('submit-button');
```

### Common Query Patterns:

```
// Get by role
screen.getByRole('button', { name: 'Save' });
screen.getByRole('textbox', { name: 'Username' });
screen.getByRole('heading', { level: 2 });

// Get by label (forms)
screen.getByLabelText('Password');
screen.getByLabelText(/email/i); // Regex

// Get by text content
screen.getByText('Hello World');
screen.getByText(/error/i); // Regex

// Get by test ID (last resort)
screen.getByTestId('unique-element');

// Async queries for data loading
await screen.findByText('Loaded'); // Waits for element
```

### FireEvent vs User Event (Ch. 11):

```
// FireEvent - lower level, less realistic
fireEvent.click(button);
fireEvent.change(input, { target: { value: 'test' } });

// User Event - higher level, more realistic
await userEvent.click(button);
await userEvent.type(input, 'test');
await userEvent.selectOptions(select, 'optionValue');
await userEvent.clear(input);
```

### Testing Form Validation:

```
test('shows validation error for invalid email', async () => {
  const user = userEvent.setup();
  render(<LoginForm />);

  const emailInput = screen.getByLabelText(/email/i);
  const submitButton = screen.getByRole('button', { name: /submit/i });

  await user.type(emailInput, 'invalid-email');
  await user.click(submitButton);

  expect(screen.getByText(/please enter a valid email/i)).toBeInTheDocument();
});
```

### Mocking Modules:

```
// Mock a module
jest.mock('./api', () => ({
  fetchUser: jest.fn().mockResolvedValue({ id: 1, name: 'John' }),
  updateUser: jest.fn().mockResolvedValue({ success: true })
}));

// Mock a single function
const mockFn = jest.fn();
jest.spyOn(Math, 'random').mockImplementation(() => 0.5);
```

## Appendix C: Deployment Platforms Comparison

| Feature | Vercel | Netlify | GitHub Pages |
|---|---|---|---|
| Free tier | Yes | Yes | Yes |
| Custom domains | Yes | Yes | Yes |
| Auto deploys | Yes | Yes | Yes |
| Preview deploys | Yes | Yes | No |
| Serverless functions | Yes | Yes | No |
| Edge functions | Yes | Yes | No |
| Analytics | Yes | Yes | No |
| CLI support | Yes | Yes | No |

**Vercel Deployment Steps:**

1. Install Vercel CLI: `npm i -g vercel`
2. Login: `vercel login`
3. Deploy: `vercel`
4. Configure: Edit vercel.json
5. Custom domain: vercel.com/dashboard → Domains

**Netlify Deployment Steps:**

1. Create netlify.toml:

```
[build]
  command = "npm run build"
  publish = "dist"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

2. Connect GitHub repo in Netlify dashboard
3. Configure build command and publish directory
4. Deploy!

**Environment Variables:**

```
// Vercel
vercel env add API_KEY

// Netlify
// Settings $\rightarrow$ Build \& deploy $\rightarrow$ Environment variables

// GitHub Actions
// Settings $\rightarrow$ Secrets $\rightarrow$ Actions
```

**Best Practices:**

- Always use production builds (minified)
- Set up cache headers for static assets
- Enable gzip/brotli compression
- Use CDN for global distribution
- Set up error boundaries for production
- Monitor performance with web vitals

**Custom Domain Setup:**

1. Purchase domain from registrar
2. Add domain in deployment platform
3. Update DNS records:
    - A record: Points to IP
    - CNAME: Points to your-app.vercel.app
4. Wait for SSL certificate (usually automatic)
5. Test with https://yourdomain.com